电商导购Agent架构演进与自进化实践分享 - 摘要

10:25 2026/10/24
上海市, CN
由AI生成，请在使用前核实

概览

本次分享深入拆解了快手电商面向C端导购Agent的架构演进与持续建设实践，重点涵盖多业务域协同、上下文压缩管理、多路用户Memory构建、轨迹诊断评测以及Bad Case自进化闭环。通过将售后与营销拆为Sub-Agent、引入独立Task Context保护层与三路Memory机制，团队有效解决了上下文膨胀和关键信息丢失等痛点，并在自研轨迹诊断与微调模型驱动下探索了自动化修复链路。

要点

- 架构由单主循环集成所有Tool升级为将策略变动高频且知识复杂的售后和营销域拆分为独立Sub-Agent以Tool形式接入。
- 上下文管理引入独立的Task Context保护层，在每轮对话后由微调小参数模型生成结构化Summary并在下一轮注入，解决关键信息丢失问题。
- 用户Memory构建采用离线2B定制模型提取特征、进线前置行为序列全量加载、在线Memory-write Tool实时写入的三路设计。
- 自动化Bad Case归因放弃无约束Codex自由诊断，转为结合LLM组件分类约束、多阶段下钻追查与ILCP强化训练的排查链路。
- 文本性质的Prompt与Tool Description支持AI自动修改与评测迭代，而Context与Memory管理策略则必须保留人工审查确认。
- 针对电商场景中效果修复导致Prompt变长、推高RT并进而拉低GMV的特殊现象，所有线上改动均强制要求通过AB实验验证。

详情

- 电商导购Agent的业务特点与挑战
  - 跨多业务域协同: 一次简单的购物Query需跨越导购域寻找SKU规格、营销域计算优惠价格以及履约域查询发货信息，且覆盖售前推荐、售中营销咨询及售后客服跟踪。 00:01:56
  - 对话长轮次与高动态性: 成交用户的平均对话轮次达30轮以上，用户需求泛化且持续纠正，对话对象与任务状态不断切换，极易在20轮后发生跑偏与牛头不对马嘴。 00:03:05
  - 对话历史信息不足: 仅凭助手内部历史对话构建Memory远远不够，需深度挖掘电商全链路触点信息并针对不同任务提供差异化检索。 00:04:18
- Agent多业务域协同架构演进与治理
  - 架构演进阶段: 去年采用预定义子图的Workflow流程；今年初升级为基于Pi模型构建的单Agent Loop架构并将各域API一键转为Tool集成。 00:06:38
  - 单Loop方案缺陷: 各域策略与接口大量汇入导致主循环上下文极度膨胀、模型能力下降；能力边界模糊且Bad Case归因困难；多业务域共同变更易引发策略冲突；各业务域技术人员亦不愿仅编写粗浅Skill而更倾向于开发具备技术价值的Sub-Agent。 00:07:38
  - Sub-Agent拆分与治理: 针对策略高频变动且知识链路复杂的业务域进行独立拆分与规范化治理。 00:09:07
    - 售后与营销拆分: 将售后和营销拆为独立的Sub-Agent，以Sub-Agent as Tool的形式接入主Agent。 00:09:16
    - 明确评测契约: 各Sub-Agent需具备独立的评测集、交付标准以及显式定义的结构化上下文传递契约。 00:09:44
    - Tool发布治理: 针对电商复杂的Tool/Skill上线建立标准CI/CD流程，严格执行发布一致性与冲突检测。 00:10:28
- 多轮对话上下文管理与压缩方案
  - 常规压缩手段局限: 针对Tool的硬规则Token截断、历史信息模型Summary摘要以及滑动窗口等常规手段，即便调整Prompt或微调摘要模型仍无法避免关

键信息丢失。 00:11:01

  - 独立Task Context保护层: 从业务语义层显式定义原始Query、对比商品及任务进度等关键上下文，独立于常规压缩之外。 00:11:34
  - 小模型循环注入: 每轮对话后由小参数模型对Task Context进行Summary，在下一次对话前重新注入模型，显著保障了关键信息的留存。 00:12:04

- 用户Memory构建与差异化检索
  - 三路数据源构建: 分别从离线、进线和在线三个维度采集并加工用户数据。 00:12:45
    - 离线提取: 基于微调的2B定制模型，从用户购买、浏览及直播间种草行为中提取固定型用户特征。 00:12:56
    - 进线序列: 采集用户进入对话框前的前置行为序列，包括商品浏览、购物车加购及直播观看记录。 00:13:07
    - 在线写入: 内置Memory-write Tool，由模型在对话过程中自主决策并实时写入。 00:13:22
  - 三层图谱存储: 建立“用户-类目-商品”三维行为图谱，利用类目层作为可枚举的标准中间层降低模型生成Tool参数的推理难度，并保留具体商品节点信息。 00:13:33
  - 差异化检索策略: 对话时全量加载用户特征与近期进线行为；将订单搜索、购物车检索及类目偏好记忆等高开销检索包装为独立Tool由Agent按需调用决策并做最终Merge。 00:14:25
- 轨迹采集诊断与评测体系建设
  - 自研轨迹诊断平台: 放弃开源组件（因跨TypeScript底层与内部Java中间件胶水层且需归因至Session和用户维度），自研兼容OpenTelemetry协议的工程采集链路，前置提取异常节点并向产研开放CI查询。 00:15:10
  - 单点能力评测框架: 在端到端评测基础上为Memory与Tool提供独立评测能力，直接复用开源Benchmark（如Memory L）验证并补齐框架短板，避免重复构建业务基准。

00:17:09

  - 基线与回归集双轨制: 评测集作为基础水位基线；另行构建覆盖高频与安全敏感问题的线上回归集，通过每日巡检与产品核对动态补充，上线要求基线波动可接受且回归集必须全部通过。 00:18:14

- Agent自进化与Bad Case闭环修复
  - Bad Case筛选机制: 规则通道抓取工具报错、完全重复调用（相同参数与链路）及显式/情绪负反馈；模型抽样聚焦超短轮次流失样本（首句离开/推荐未承接）及导购超长轮次未成交样本。 00:21:38
  - 约束归因链路升级: 摒弃自由放开代码与轨迹导致的不稳定归因（如面对超10轮死循环仅建议调大轮次或增加不要重复的System Prompt，无法定位Tool Description定义模糊）、泛滥添加System Reminder及解释性差问题，升级为LLM组件分类约束、多阶段下钻追查与归因合并流程，实验证明结合LLaMA微调与ILCP训练诊断提升显著。 00:23:32
  - 沙箱改动与审查边界: Codex在容器云独立环境分支中运行修改并保留全流程留痕，放开Prompt与Tool Description的自动修改迭代，而Context与Memory管理策略修改必须强制人工审查确认。 00:27:41
  - 验证与线上AB实验: 修复改动须在独立Bad Case集上验证且不能引发整体基线退化；由于Prompt修补过细会膨胀上下文、拉长推理RT并在C端敏感场景下拉低GMV，所有改动均须通过线上AB实验验证无负向波动方可发布。 00:28:52
  - 实践局限与边界探索: 工程Bug类可快速自动化修复，Prompt/Tool优化需多轮收敛，而Memory加工侧策略缺失证据链难以自动修复；针对连续两轮相同参数调用失败的死循环，仍需权衡是由Harness显式注入报错Hint促成纠错，还是依赖更大基模自纠正。 00:31:00
- 问答环节关于架构实现与上下文优化的讨论
  - 多模块复合Bad Case: 针对多模块共同引发的Bad Case，分类环节需提供更全面的上下文以识别多模块问题，修复环节同样需支持多模块协同修复。 00:35:21
  - 底层架构分工: 仅在Agent Loop层依赖Pi相关开源组件负责循环维护与LLM Pr

ovider对接，外层的Agent代理、服务对接、上下文压缩管理及Memory检索均完全自研。 00:36:53

- 混合检索实现: Memory实际落地结合了向量检索、标签检索及行为图谱检索等多路手段，检索效果核心取决于Memory本身的业务结构设计。 00:37:59
- 多粒度商品检索与冲突消除: 检索Tool按一级类目与二级属性分级设计；上下文冲突主要通过Task Context的结构化约束进行天然规避，历史Summary压缩与Task Context压缩则分别采用针对具体场景微调的不同小参数模型剔除冲突。 00:40:55
- 线上RT主要瓶颈: 相比多轮Tool Call的生成开销，多轮长对话后上下文持续积压膨胀所导致的推理时延增长，才是当前C端链路最严峻的RT瓶颈。 00:43:05

后续行动

关注下一场关于线上故障处理与24小时响应机制的分享。